

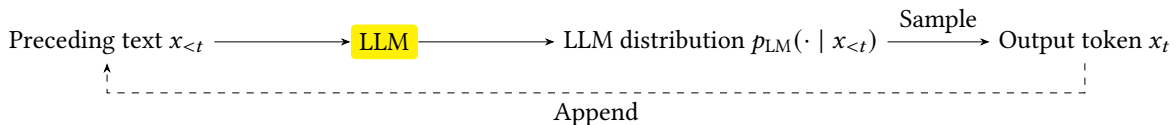
Υδατογράφηση Κειμένου

Ήδη, πολλά από τα κείμενα που βλέπουμε καθημερινά παράγονται από Μεγάλα Γλωσσικά Μοντέλα (Large-Language Models, LLMs) και το πιθανότερο είναι ότι όσο περνάει ο καιρός ολοένα και περισσότερα από τα κείμενα που βλέπουμε θα παράγονται από μηχανές. Ευκαταίο είναι, λοιπόν, να μπορούμε να γνωρίζουμε αν κάποιο κείμενο που συναντάμε είναι παραγόμενο από μηχανή, ή αν έχει γραφτεί από, ή έχει επωφεληθεί από, ανθρώπινο χέρι.

Πέρα από την απλή επιθυμία να ξέρουμε τι κείμενο διαβάζουμε, ανθρώπου ή μηχανής, η δυνατότητα διάκρισης μεταξύ των δύο είναι και νομική απαίτηση, την οποία θεσπίζει το Άρθρο 50 της Πράξης της Ευρωπαϊκής Ένωσης για την Τεχνητή Νοημοσύνη (EU AI Act). Σύμφωνα με το άρθρο, το οποίο τέθηκε σε εφαρμογή στις 2 Αυγούστου 2026, οι πάροχοι γλωσσικών μοντέλων πρέπει να εφαρμόζουν στο υλικό που παράγουν *υδατογραφήματα* (watermarks), μηχανικώς αναγνώσιμα (machine readable). Αυτά πρέπει να είναι αόρατα, με την έννοια ότι δεν αλλοιώνουν το νόημα του κειμένου, ή το περιεχόμενο μιας εικόνας, στην περίπτωση εικόνων. Μία γνωστή προσέγγιση υδατογράφησης είναι το SynthID-Text, που αναπτύχθηκε από ερευνητές της Google DeepMind [1].

Δημιουργία του Υδατογράμματος

Για να δούμε πώς μπορούμε να υδατογραφήσουμε κείμενο, ας ξεκινήσουμε από τον τρόπο με τον οποίο ένα γλωσσικό μοντέλο παράγει την έξοδο που μας δίνει. Το κείμενο παράγεται τεμάχιο (token) προς τεμάχιο. Για να παράξει το επόμενο τεμάχιο x_t , το μοντέλο παίρνει ως είσοδο το κείμενο που έχει ήδη παραγάγει, $x_{<t}$, και υπολογίζει την πιθανότητα κάθε πιθανού τεμαχίου του λεξιλογίου του να είναι το επόμενο τεμάχιο x_t . Δηλαδή, δημιουργεί μια κατανομή $p_{LM}(\cdot \mid x_{<t})$ που περιέχει την πιθανότητα κάθε πιθανού τεμαχίου. Στη συνέχεια, δειγματοληπτεί το επόμενο τεμάχιο με βάση τις πιθανότητες των τεμαχίων σε αυτήν την κατανομή. Αφού παράξει το τεμάχιο, το προσαρτά στο κείμενο και ο βρόχος ξεκινά από την αρχή [1]:

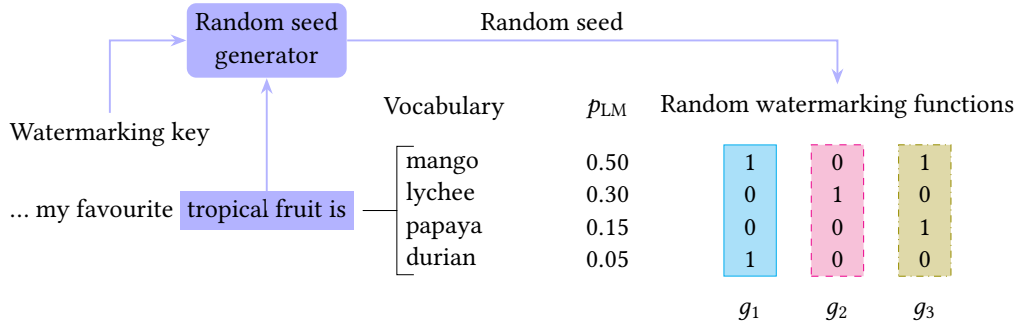


Το SynthID-Text εισάγει μια λεπτή αλλαγή στη διαδικασία. Αντί να δειγματοληπτεί από την κατανομή, εισάγει έναν αλγόριθμο δειγματοληψίας που εξαρτάται από έναν τυχαίο σπόρο (seed) r_t για κάθε τεμάχιο x_t . Αυτό επηρεάζει το αποτέλεσμα της δειγματοληψίας, αλλά αν γίνει με προσοχή, η αλλαγή στην έξοδο δεν θα αλλοιώσει το κείμενο με τρόπο που θα μπορούσε να εντοπίσει ένας άνθρωπος. Ταυτόχρονα όμως, αν έχουμε ένα κείμενο, τον αλγόριθμο δειγματοληψίας, και τους τυχαίους σπόρους, είναι δυνατόν να εντοπίσουμε ότι το κείμενο παρήχθη από γλωσσικό μοντέλο. Γιατί; Επειδή τα τεμάχια διαφέρουν ελαφρώς από αυτά που θα ήταν με απλή δειγματοληψία· μετρώντας τη διαφορά, μπορούμε να συμπεράνουμε πόσο πιθανό είναι ένα κείμενο να έχει παραχθεί από το μοντέλο. Η υδατογράφηση κειμένου, λοιπόν, είναι η εισαγωγή των ελαφρών αυτών αλλαγών στο κείμενο οι οποίες, ενώ δεν είναι εντοπίσιμες από ανθρώπους, είναι εντοπίσιμες από μηχανές.

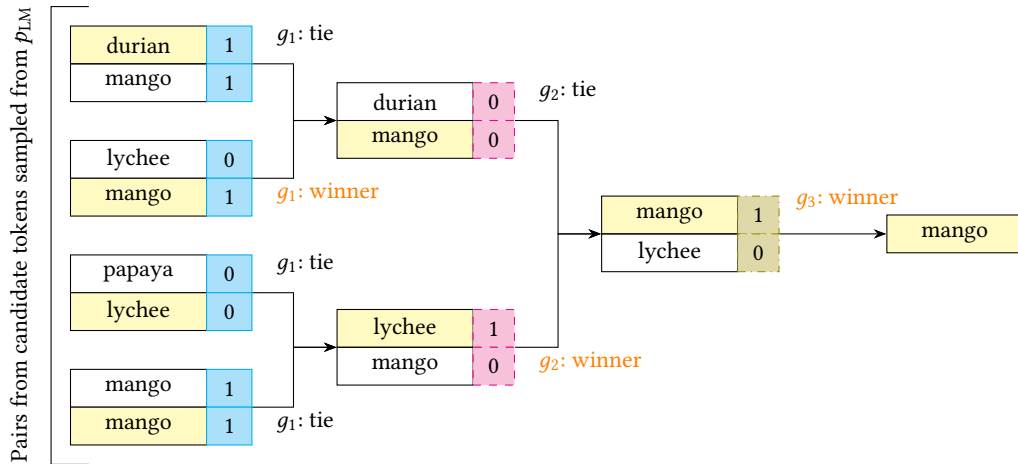
Στην πραγματικότητα, το μοντέλο δεν χρησιμοποιεί κατ' ανάγκη όλη την έξοδο που έχει ήδη παραγάγει για να παραγάγει το επόμενο τεμάχιο. Ας υποθέσουμε ότι το μοντέλο έχει παραγάγει τη φράση «το αγαπημένο μου τροπικό φρούτο είναι». Από τη φράση αυτή θα κρατήσει ένα τμήμα από το τέλος, ας

πούμε «τροπικό φρούτο είναι», το οποίο είναι τα *συμφραζόμενα* (context) που χρησιμοποιούνται για να παραχθεί το επόμενο τεμάχιο. Στη συνέχεια θα δημιουργήσει μια κατανομή πιθανότητας πάνω σε όλα τα πιθανά τεμάχια. Αν τα πιθανότερα επόμενα τεμάχια είναι «μάνγκο» με πιθανότητα $p_{LM} = 0.50$, «λίτσι» με πιθανότητα $p_{LM} = 0.30$, «παπάγια» με πιθανότητα $p_{LM} = 0.15$, και «ντούριαν», με πιθανότητα $p_{LM} = 0.05$, το μοντέλο κανονικά θα δειγματοληπτούσε ένα τεμάχιο τυχαία με βάση αυτές τις πιθανότητες.

Αντί γι' αυτό, το μοντέλο διεξάγει ένα μικρό τουρνουά ανάμεσα σε υποψήφια τεμάχια. Η διαδικασία ονομάζεται *Δειγματοληψία Τουρνουά* (Tournament Sampling), και χρησιμοποιεί έναν αριθμό τυχαίων συναρτήσεων υδατογράφησης. Παίρνουμε τον τυχαίο σπόρο r_t και τον δίνουμε σε έναν αριθμό m συναρτήσεων υδατογράφησης $g_1, g_2, \dots, g_m$. Αυτές είναι ανεξάρτητες ψευδοτυχαίες συναρτήσεις διατεταγμένες σε επίπεδα, οι οποίες επιστρέφουν μια δυαδική τιμή, 0 ή 1, όταν καλούνται ως $g_\ell(x_t, r_t)$ για οποιοδήποτε υποψήφιο τεμάχιο x_t , και οποιοδήποτε επίπεδο ℓ από 1 έως m . Δείτε την ακόλουθη εικόνα για $m = 3$ [1].



Το τουρνουά ξεκινά δειγματοληπτώντας με επανατοποθέτηση $M = 2^m$ υποψήφια τεμάχια από την κατανομή του μοντέλου $p_{LM}(\cdot | x_{<t})$. Αυτά μπαίνουν σε ένα τουρνουά m επιπέδων. Χωρίζουμε τους συμμετέχοντες σε $M/2$ ζεύγη. Στο πρώτο επίπεδο του τουρνουά, επιλέγουμε από κάθε ζεύγος το τεμάχιο με την υψηλότερη τιμή υπό την $g_1(\cdot, r_t)$. Σε περίπτωση ισοπαλίας, διαλέγουμε τυχαία ένα από το ζεύγος. Μας μένουν $M/2$ συμμετέχοντες, τους οποίους ομαδοποιούμε τυχαία σε $M/4$ ζεύγη. Το δεύτερο επίπεδο του τουρνουά λειτουργεί όπως το πρώτο: η $g_2(\cdot, r_t)$ επιλέγει πάλι τους νικητές, με τις ισοπαλίες να λύνονται τυχαία. Συνεχίζουμε με τα υπόλοιπα επίπεδα, ώσπου ένα μόνο τεμάχιο αναδεικνύεται τελικός νικητής, το x_t , όπως φαίνεται στη συνέχεια [1].



Οι συναρτήσεις υδατογράφησης $g_\ell(x_t, r_t)$ παίρνουν ως είσοδο τον σπόρο r_t , οπότε μόνο όποιος έχει τον σπόρο μπορεί να ξαναυπολογίσει τις τιμές g . Αυτό είναι σημαντικό, γιατί διαφορετικά οποιοσδήποτε μπορεί να υπολογίσει την g μπορεί να παραβιάζει την προσέγγιση και προς τις δύο μεριές: μπορεί να

αφαιρέσει ένα υδατογράφημα, ξαναγράφοντας το κείμενο ώστε να προτιμά τεμάχια με $g = 0$, έτσι ώστε κείμενο παραγόμενο από μηχανή να περνά για ανθρώπινο, ή να πλαστογραφήσει ένα υδατογράφημα, επιλέγοντας τεμάχια με $g = 1$ έτσι ώστε ανθρώπινο κείμενο να φαίνεται ότι έχει παραχθεί από μηχανή.

Υπάρχει όμως ένα πρόβλημα με τη Δειγματοληψία Τουρνουά όπως την περιγράψαμε. Καθώς δειγματοληπτεί $M = 2^m$ τεμάχια, η χρήση του δεν μπορεί να κλιμακωθεί. Για $m = 12$, θα δειγματοληπτήσει $2^{12} = 4096$ τεμάχια για να ξεκινήσει το τουρνουά. Για $m = 30$, θα χρειαστεί να δειγματοληπτήσει πάνω από ένα δισεκατομμύριο τεμάχια. Αυτός μάλιστα είναι ο προεπιλεγμένος αριθμός επιπέδων που χρησιμοποιεί το SynthID-Text, αλλά προφανώς η δειγματοληψία πάνω από ενός δισεκατομμυρίου τεμαχίων δεν είναι πρακτική.

Ευτυχώς, μπορούμε να υλοποιήσουμε τη Δειγματοληψία Τουρνουά με πολύ πιο αποδοτικό τρόπο. Ας δούμε τι συμβαίνει σε ένα επίπεδο του τουρνουά. Ένας αγώνας παίρνει δύο υποψηφίους και επιστρέφει έναν. Ο κάθε ένας από τους δύο υποψηφίους προκύπτει ανεξάρτητα με δειγματοληψία από την κατανομή του μοντέλου, την οποία από εδώ και πέρα θα γράφουμε απλώς p αντί για $p_{LM}(\cdot \mid x_{<t})$ · ας ονομάσουμε τους δύο υποψηφίους X και Y . Και το X και το Y είναι τεμάχια, και το καθένα έχει τιμή g 0 ή 1.

Έστω A το σύνολο των τεμαχίων με $g = 0$ και $q_A = \sum_{x_t \in A} p(x_t)$ η συνολική πιθανότητα των τεμαχίων αυτών· ορίζουμε αντίστοιχα το B και το q_B για $g = 1$. Κάθε τεμάχιο ανήκει σε ακριβώς ένα από τα A ή B , οπότε:

$$q_A + q_B = 1$$

Ο κανόνας είναι ότι κερδίζει η υψηλότερη τιμή g . Για τις ισοπαλίες είπαμε πιο πάνω ότι τις λύνουμε τυχαία. Προς το παρόν ας υιοθετήσουμε έναν απλό κανόνα, δίνοντας κάθε ισοπαλία στον αριστερό υποψήφιο, τον X . Το κάνουμε αυτό επειδή κάνει την ανάλυση που ακολουθεί πολύ ευκολότερη. Αυτό δεν επηρεάζει τη γενικότητα: στο τέλος αυτής της ενότητας θα δούμε ότι οι δύο κανόνες ισοπαλίας παράγουν ακριβώς την ίδια κατανομή, οπότε είμαστε ελεύθεροι να διαλέξουμε όποιον μας βολεύει.

Καθώς οι τιμές g είναι δυαδικές, 0 ή 1, το να δίνουμε τις ισοπαλίες στα αριστερά σημαίνει ότι ο νικητής είναι ο X , εκτός αν $g(X) = 0$ και $g(Y) = 1$. Έτσι, για οποιοδήποτε τεμάχιο x_t , υπάρχουν ακριβώς δύο αμοιβαίως αποκλειόμενοι τρόποι με τους οποίους μπορεί να βγει από τον αγώνα:

$$\Pr[\text{νικητής} = x_t] = p'(x_t) = \underbrace{\Pr[X = x_t \text{ και } g(Y) \leq g(X)]}_{\text{το } x_t \text{ κερδίζει ως αριστερός υποψήφιος}} + \underbrace{\Pr[Y = x_t \text{ και } g(Y) = 1 \text{ και } g(X) = 0]}_{\text{το } x_t \text{ κερδίζει ως δεξιός υποψήφιος}}$$

Ας πάρουμε τώρα με τη σειρά τις δύο πιθανές τιμές g του x_t .

Αν $g(x_t) = 0$, για τον πρώτο όρο, το $X = x_t$ συμβαίνει με πιθανότητα $p(x_t)$, και χρειαζόμαστε $g(Y) = 0$, δηλαδή $Y \in A$, που συμβαίνει με πιθανότητα q_A . Δεν χρειάζεται το X να νικήσει πραγματικά το Y : χάρη στον κανόνα της ισοπαλίας, αρκεί να έρθει ισόπαλο μαζί του. Τα ενδεχόμενα είναι ανεξάρτητα, οπότε ο όρος γίνεται $p(x_t)q_A$. Ο δεύτερος όρος απαιτεί $g(x_t) = 1$, που εδώ δεν ισχύει, οπότε είναι μηδέν. Επομένως παίρνουμε:

$$p'(x_t) = p(x_t)q_A$$

Αν $g(x_t) = 1$, για τον πρώτο όρο έχουμε πάλι $X = x_t$ με πιθανότητα $p(x_t)$, και το $g(Y)$ δεν μας ενδιαφέρει καθόλου: αν $g(Y) = 0$ τότε το X κερδίζει κατευθείαν, και αν $g(Y) = 1$ έχουμε ισοπαλία, την οποία

κερδίζει ο αριστερός υποψήφιος. Άρα ο όρος είναι $p(x_t) \cdot 1 = p(x_t)$: με άλλα λόγια, ένα τεμάχιο με $g = 1$ τοποθετημένο στα αριστερά δεν μπορεί να χάσει. Για τον δεύτερο όρο, $Y = x_t$ με πιθανότητα $p(x_t)$ και $X \in A$ με πιθανότητα q_A , που δίνει $p(x_t) q_A$. Συνολικά:

$$p'(x_t) = p(x_t) + p(x_t)q_A = p(x_t)(1 + q_A) = p(x_t)(2q_A + q_B),$$

όπου το τελευταίο βήμα χρησιμοποιεί ότι $1 = q_A + q_B$.

Συνδυάζοντας τα δύο, παίρνουμε:

$$p'(x_t) = \begin{cases} p(x_t) q_A & \text{if } g(x_t) = 0 \\ p(x_t) (2q_A + q_B) & \text{if } g(x_t) = 1 \end{cases}$$

Η πρώτη περίπτωση σημαίνει ότι ένα τεμάχιο στο A κερδίζει μόνο από τα αριστερά, και μόνο απέναντι σε ένα άλλο μέλος του A . Η δεύτερη περίπτωση σημαίνει ότι ένα τεμάχιο στο B έχει δύο ευκαιρίες: κερδίζει από τα αριστερά πάντα, $p(x_t)(q_A + q_B)$, και από τα δεξιά όποτε ο αντίπαλός του βρίσκεται στο A , $p(x_t)q_A$. Μπορούμε να το δούμε αυτό στον ακόλουθο πίνακα:

X	Y	πιθανότητα	ποιος κερδίζει
A	A	q_A^2	ένα τεμάχιο του A : ισοπαλία, άρα το αριστερό
A	B	$q_A q_B$	το τεμάχιο του B , κατευθείαν
B	A	$q_B q_A$	το τεμάχιο του B , κατευθείαν
B	B	q_B^2	ένα τεμάχιο του B : ισοπαλία, άρα το αριστερό

Προσθέτοντας τα τέσσερα διαφορετικά αποτελέσματα παίρνουμε $q_A^2 + 2q_A q_B + q_B^2$, που είναι $(q_A + q_B)^2 = 1$, όπως πρέπει, αφού μια κατανομή πρέπει να αθροίζει στο 1.

Ο ίδιος κανόνας μπορεί να γραφτεί χωρίς διάκριση περιπτώσεων, αφού $q_A = 1 - q_B$:

$$p'(x_t) = p(x_t) [1 + g(x_t) - q_B]$$

Αυτό συμβαίνει στο πρώτο επίπεδο του τουρνουά. Τι γίνεται με τα υπόλοιπα $m - 1$;

Πιο πάνω μόλις είδαμε ότι ένας αγώνας μετατρέπει δύο ανεξάρτητες λήψεις από την p σε μία λήψη από την p' . Το πρώτο επίπεδο του τουρνουά είναι $M/2$ τέτοιοι αγώνες, που διεξάγονται σε διακριτά μεταξύ τους ζεύγη από τους M υποψηφίους. Ισοδύναμα, αντί να διεξάγουμε αγώνες, μπορούμε να έχουμε $M/2$ ανεξάρτητες λήψεις από την p' . Έτσι, μετά το πρώτο επίπεδο κρατάμε $M/2$ ανεξάρτητες λήψεις από την p' , που είναι η κατάσταση από την οποία ξεκινήσαμε, με τους μισούς συμμετέχοντες και με την p' αντί για την p . Το δεύτερο επίπεδο κάνει επομένως το ίδιο πράγμα ξανά, με την g_2 αντί για την g_1 , δίνοντας $M/4$ ανεξάρτητες λήψεις από την p'' , και ούτω καθεξής.

Μετά από m επίπεδα μένει ένας μόνο συμμετέχων, και η κατανομή του είναι η p όπως έχει προκύψει μετά από m ενημερώσεις, μία ανά επίπεδο, με τη σειρά. Ας συμβολίσουμε την p' ως $p^{(1)}$, την p'' ως $p^{(2)}$, μέχρι την $p^{(m)}$, τότε:

$$p \xrightarrow{g_1} p^{(1)} \xrightarrow{g_2} p^{(2)} \longrightarrow \dots \xrightarrow{g_m} p^{(m)}$$

Όλα αυτά σημαίνουν ότι, αντί να διεξάγουμε αγώνες, αρκεί να υπολογίσουμε την κατανομή m φορές, μία για κάθε επίπεδο, κάθε φορά με τη δική του συνάρτηση g , και στο τέλος να δειγματοληπτήσουμε ένα

μόνο τεμάχιο από την κατανομή $p^{(m)}$ που προκύπτει. Σε επαναλαμβανόμενη δειγματοληψία τα τεμάχια βγαίνουν με ακριβώς τις ίδιες πιθανότητες όπως και με την αρχική προσέγγιση, και η διαδικασία δεν εξαρτάται πλέον από το $M = 2^m$.

Τι κερδίζουμε από αυτό; Η διεξαγωγή του τουρνουά σημαίνει κλήρωση $M = 2^m$ υποψηφίων και μετά διεξαγωγή $2^m - 1$ αγώνων νοκ-άουτ. Καθεμιά από αυτές τις κληρώσεις σημαίνει τη μετατροπή ενός τυχαίου αριθμού σε τεμάχιο. Μπορούμε να το κάνουμε αυτό διαλέγοντας έναν τυχαίο αριθμό $0 \leq u < 1$ και διατρέχοντας μετά την αθροιστική κατανομή πάνω στο λεξιλόγιο V , το σύνολο των διαφορετικών τεμαχίων, ώσπου το τρέχον άθροισμα να ξεπεράσει το u . Το κόστος τότε είναι $O(2^m |V|)$. Θα μπορούσαμε να βελτιώσουμε λίγο τα πράγματα κάνοντας δυαδική αναζήτηση στην αθροιστική κατανομή, φτάνοντας σε κόστος $O(2^m \lg |V|)$ —αυτό δεν θα μας σώσει. Ο υπολογισμός της κατανομής με τη διαστρωματωμένη προσέγγιση σημαίνει m περάσματα πάνω από το λεξιλόγιο. Σε κάθε πέραςμα υπολογίζουμε τα αθροίσματα q_A και q_B , που απαιτούν $|V|$ προσθέσεις, και μετά κάνουμε έναν πολλαπλασιασμό ανά τεμάχιο, που απαιτεί $|V|$ πολλαπλασιασμούς. Μετά το τελευταίο πέραςμα κάνουμε μία μόνο κλήρωση. Αυτό δίνει μόνο γραμμικό κόστος $O((2m + 1)|V|) = O(m|V|)$, έναντι του εκθετικού κόστους των αγώνων νοκ-άουτ.

Υπάρχει και κάτι ακόμη. Πρέπει να πάρουμε τους M υποψηφίους και μετά να τους βάλουμε σε ζεύγη, οπότε πρέπει να αποθηκεύσουμε και τους 2^m . Στο $m = 30$, αυτό είναι πάνω από ένα δισεκατομμύριο. Μπορεί να ξεμεινουμε από μνήμη προτού ξεμεινουμε από χρόνο.

Ας επιστρέψουμε τώρα στο θέμα των ισοπαλιών. Ας υποθέσουμε ότι τις λύνουμε με ρίψη νομίσματος αντί να τις δίνουμε στα αριστερά. Ποια θα είναι η πιθανότητα δειγματοληψίας του τεμαχίου x_t ;

Πρώτα, ας πάρουμε ένα τεμάχιο με $g(x_t) = 1$: κερδίζει κατευθείαν όποτε ο αντίπαλός του βρίσκεται στο A , που μπορεί να συμβεί με το x_t σε οποιαδήποτε από τις δύο πλευρές, δίνοντας $2 p(x_t) q_A$. Όταν ο αντίπαλός του βρίσκεται στο B , το τεμάχιο κερδίζει τις μισές φορές, συνεισφέροντας $2 \cdot \frac{1}{2} p(x_t) q_B = p(x_t) q_B$. Το σύνολο είναι $p(x_t)(2q_A + q_B)$, το ίδιο όπως πριν.

Έπειτα, ας πάρουμε $g(x_t) = 0$. Το τεμάχιο μπορεί να κερδίσει μόνο με ισοπαλία, σε οποιαδήποτε από τις δύο πλευρές, τις μισές φορές: $2 \cdot \frac{1}{2} p(x_t) q_A = p(x_t) q_A$, επίσης το ίδιο όπως πριν. Οι δύο κανόνες ισοπαλίας είναι ισοδύναμοι.

Τυχαιότητα

Η Δειγματοληψία Τουρνουά απαιτεί μια ακολουθία τυχαίων σπόρων $r_1, r_2, \dots, r_T$, αφού χρησιμοποιούμε έναν τυχαίο σπόρο r_t για κάθε τεμάχιο. Χρειαζόμαστε επομένως έναν τρόπο να παράγουμε αυτούς τους τυχαίους σπόρους, δηλαδή χρειαζόμαστε μια γεννήτρια τυχαίων σπόρων. Ένας απλός τρόπος να το κάνουμε αυτό είναι να χρησιμοποιήσουμε μια ντετερμινιστική συνάρτηση f_r που παίρνει ως είσοδο την ακολουθία των τεμαχίων $x_{<t} = x_1, x_2, \dots, x_{t-1}$ και ένα κλειδί υδατογράφησης k και βγάζει έναν τυχαίο σπόρο $r_t = f_r(x_{<t}, k)$. Το SynthID-Text χρησιμοποιεί για την f_r μια προσέγγιση κυλιόμενου παραθύρου, στην οποία εφαρμόζουμε μια συνάρτηση κατακερματισμού h στα τελευταία H τεμάχια, τα συμφραζόμενα, και στο κλειδί k : $f_r(x_{<t}, k) = h(x_{t-H}, x_{t-H+1}, \dots, x_{t-1}, k)$.

Αφού πάρουμε τον τυχαίο σπόρο, τον τροφοδοτούμε σε m τυχαίες συναρτήσεις, μία ανά επίπεδο, τις $g_\ell(x, r_t)$ για $\ell = 1, 2, \dots, m$ και για κάθε τεμάχιο x του λεξιλογίου. Οι συναρτήσεις g_ℓ επιστρέφουν τιμές g που είναι δυαδικές, 0 ή 1, με ίση πιθανότητα, όπως είδαμε πιο πάνω.

Αντί για τη διαδικασία δύο βημάτων της παραγωγής σπόρων και της τροφοδότησής τους στις συναρτήσεις g_ℓ , μπορούμε να χρησιμοποιήσουμε μία μόνο συνάρτηση που παίρνει ως είσοδο το κλειδί υδατογράφησης, το παράθυρο συμφραζομένων, τον δείκτη του τρέχοντος επιπέδου, και το τεμάχιο, και βγάζει 0 ή 1 με ίση πιθανότητα. Σε Python, αυτό γίνεται απλά ως εξής:

```
payload = f"{{key}}|{'|'.join(window)}}|{{layer}}|{{token}}"
g = int.from_bytes(sha256(payload.encode()).digest(), "big") & 1
```

Το κλειδί συνδέει ένα υδατογράφημα με τον παραγωγό του. Διαφορετικοί πάροχοι θα έχουν διαφορετικά κλειδιά, οπότε τα υδατογραφήματά τους θα διαφέρουν, ακόμη και για το ίδιο περιεχόμενο. Μόνο με το ίδιο το κλειδί ενός παρόχου μπορούμε να ανιχνεύσουμε τα υδατογραφήματα στα κείμενά του. Το κλειδί πρέπει να παραμένει μυστικό: αν ήταν δημόσιο, οποιοσδήποτε θα μπορούσε να πλαστογραφήσει την προέλευση, σημαδεύοντας το δικό του κείμενο ως κείμενο άλλου παρόχου.

Για να παραγάγουμε μια τιμή g πρέπει να έχουμε προχωρήσει πέρα από τα πρώτα H τεμάχια· συνεπώς, η υδατογράφηση μπορεί να ξεκινήσει μόνο αφού έχουν παραχθεί αυτά τα H τεμάχια. Σε ένα πραγματικό περιβάλλον, προέρχονται από την προτροπή (prompt) του χρήστη. Το δικό μας μοντέλο δεν έχει χρήση, οπότε τα επινοεί, κληρώνοντας H τεμάχια ομοιόμορφα από το λεξιλόγιο· θα τα ονομάζουμε κι εμείς προτροπή ούτως ή άλλως. Δεν φέρουν υδατογράφημα, και, μην έχοντας δικό τους παράθυρο συμφρασμένων, δεν βαθμολογούνται ποτέ. Επίσης, έχει αποδειχθεί ότι αν το κείμενο περιέχει επαναλαμβανόμενες ακολουθίες, πράγμα που σημαίνει ότι θα συναντήσουμε τα ίδια H τεμάχια επανειλημμένα, τότε το υδατογράφημα μπορεί να εισαγάγει μια επαναλαμβανόμενη μεροληψία που θα επηρεάσει την ποιότητα του παραγόμενου κειμένου. Για να το αποφύγουμε αυτό, το υδατογράφημα πρέπει να εφαρμόζεται μόνο σε διαφορετικά κυλιόμενα παράθυρα μήκους H . Αυτό ακριβώς κάνει το SynthID-Text. Η δική μας γεννήτρια δεν το κάνει, χάριν απλότητας: διεξάγει το τουρνουά σε κάθε θέση πέρα από την προτροπή. Εφαρμόζουμε τον κανόνα κατά των επαναλαμβανόμενων παραθύρων μόνο κατά την ανίχνευση του υδατογραφήματος, έτσι ώστε μια θέση της οποίας το παράθυρο έχει ήδη εμφανιστεί νωρίτερα στο κείμενο να παρακάμπτεται. Και αυτό μας φέρνει στην ανίχνευση του υδατογραφήματος.

Ανίχνευση Υδατογραφήματος

Η Δειγματοληψία Τουρνουά επιλέγει από την κατανομή του μοντέλου τεμάχια που είναι πιθανό να λάβουν υψηλότερες τιμές από τις τυχαίες συναρτήσεις υδατογράφησης $g_\ell(\cdot, r_t)$. Επομένως, για να ανιχνεύσουμε αν κάποιο κείμενο που αποτελείται από τα τεμάχια $x_1, x_2, \dots, x_T$ είναι υδατογραφημένο, πρέπει να μετρήσουμε πόσο υψηλά βαθμολογείται το κείμενο ως προς τις συναρτήσεις αυτές. Ο ανιχνευτής υδατογραφήματος, λοιπόν, ξαναυπολογίζει τις τιμές g . Σε κάθε θέση πέρα από τα πρώτα H τεμάχια ξαναφτιάχνει το παράθυρο των προηγούμενων H τεμαχίων. Αν το παράθυρο αυτό έχει ξαναεμφανιστεί, η θέση παρακάμπτεται· διαφορετικά η θέση είναι *βαθμολογήσιμη*, και ο ανιχνευτής κατακεραματίζει το παράθυρο μαζί με το κλειδί και το επίπεδο για να υπολογίσει την $g_\ell(x_t, r_t)$ για το τεμάχιο που εμφανίζεται πραγματικά εκεί.

Θα συμβολίζουμε με $\hat{T}$ το σύνολο των βαθμολογήσιμων θέσεων. Η βαθμολογία ενός κειμένου d είναι ο μέσος όρος στις θέσεις αυτές και στις συναρτήσεις υδατογράφησης:

$$\text{Score}(d) = \frac{1}{m|\hat{T}|} \sum_{t \in \hat{T}} \sum_{\ell=1}^m g_\ell(x_t, r_t)$$

Ας υποθέσουμε τώρα ότι μας δίνεται ένα υδατογραφημένο κείμενο για έλεγχο. Κατά την υδατογράφηση, το τουρνουά επέλεξε τεμάχια με $g = 1$ (εκτός από τις περιπτώσεις όπου και οι δύο υποψήφιοι σε έναν αγώνα είχαν $g = 0$), οπότε οι ξαναυπολογισμένες τιμές g τείνουν προς το 1 και η βαθμολογία ανεβαίνει πάνω από 0,5. Αντίθετα, αν μας δοθεί κείμενο που δεν έχει υδατογραφηθεί, οι ξαναυπολογισμένες τιμές g είναι αμερόληπτες ρίψεις νομίσματος και η βαθμολογία θα είναι περίπου 0,5. Η ανίχνευση, λοιπόν, είναι η διάκριση ανάμεσα σε αυτές τις δύο καταστάσεις.

Για να το κάνουμε αυτό, ξεκινάμε τρέχοντας τον βαθμολογητή πάνω σε μη υδατογραφημένα κείμενα, ένα σύνολο κειμένων βαθμονόμησης (calibration). Υπολογίζουμε ένα κατώφλι βαθμολογίας που αντιστοιχεί στο ανώτερο $\alpha\%$ των υψηλότερων βαθμολογιών· π.χ., $\alpha = 1\%$. Καθώς πρόκειται για μη υδατογραφημένα κείμενα, το να βάλουμε το κατώφλι στο α σημαίνει ότι το $\alpha\%$ των μη υδατογραφημένων κειμένων ανιχνεύονται εσφαλμένα ως υδατογραφημένα. Επομένως, το $\alpha\%$ είναι το Ποσοστό Ψευδώς Θετικών (False-Positive Rate, FPR). Για να εκτιμήσουμε την απόδοση του ανιχνευτή μας, μετράμε επίσης το ποσοστό των υδατογραφημένων κειμένων που βαθμολογούνται πάνω από αυτό το κατώφλι. Αυτό, λοιπόν, είναι το Ποσοστό Αληθώς Θετικών (True-Positive Rate, TPR) για σταθερό Ποσοστό Ψευδώς Θετικών (FPR) $\alpha\%$ και τον συμβολίζουμε με $TPR@FPR = \alpha\%$.

Για να υπολογίσουμε το κατώφλι, πρέπει να βρούμε το α ποσοστημόριο της κατανομής των βαθμολογιών των κειμένων. Αυτό είναι απλό αν όλα τα κείμενα έχουν τον ίδιο αριθμό βαθμολογήσιμων θέσεων. Ο παρονομαστής της $Score(d)$ είναι τότε ο ίδιος για όλα τα κείμενα, οπότε μπορούμε να τον αγνοήσουμε και να δουλέψουμε μόνο με τους αριθμητές, επειδή είναι ευκολότερο να δουλεύουμε με ακέραιους. Για να βρούμε το κατώφλι, ξεκινάμε δημιουργώντας ένα ιστόγραμμα που κρατά τα πλήθη των αριθμητών των κειμένων. Το ιστόγραμμα μπορεί να υλοποιηθεί ως πίνακας μεγέθους $mL + 1$, όπου L το κοινό μήκος των κειμένων. Το μέγεθος αυτό είναι πάνω από τον μεγαλύτερο δυνατό αριθμητή, που είναι $m(L - H)$, αφού οι πρώτες H θέσεις δεν βαθμολογούνται ποτέ. Για κάθε κείμενο υπολογίζουμε τον αριθμητή της βαθμολογίας του:

$$k = \sum_{t \in \hat{T}} \sum_{\ell=1}^m g_{\ell}(x_t, r_t)$$

και προσθέτουμε ένα στο κελί του ιστογράμματος στη θέση k .

Σημειώστε ότι δεν έχει νόημα να υπολογίσουμε κατώφλι σε κείμενα διαφορετικού μήκους. Ο αριθμητής μεγαλώνει με το μήκος του κειμένου, πράγμα που θα έκανε ένα ιστόγραμμα πάνω σε ανάμεικτα μήκη ιστόγραμμα με μη συγκρίσιμες ποσοότητες. Γενικότερα, ένας αριθμητής είναι συγκρίσιμος μόνο ανάμεσα σε κείμενα που βαθμολογήθηκαν πάνω στον ίδιο αριθμό θέσεων. Όταν κανένα παράθυρο δεν επαναλαμβάνεται, ένα κείμενο μήκους L έχει ακριβώς $L - H$ από αυτές, οπότε ίσα μήκη δίνουν ίσο αριθμό θέσεων· οτιδήποτε άλλο στερεί από ένα κείμενο κάποιες από τις θέσεις του χαλάει τη σύγκριση. Μολονότι η βαθμολογία η ίδια δεν μεγαλώνει με το μήκος (είναι μέσος όρος, και το μη υδατογραφημένο κείμενο βαθμολογείται περίπου στο 0,5 ανεξάρτητα από το μήκος του), η διασπορά της βαθμολογίας μικραίνει όσο τα κείμενα μεγαλώνουν.

Αφού περάσουμε από όλα τα κείμενα και συμπληρώσουμε το ιστόγραμμα, διατρέχουμε τα κελιά του ιστογράμματος από το τέλος του προς την αρχή του, προσθέτοντας τα πλήθη καθώς προχωράμε. Συσσωρεύουμε επομένως τον αριθμό των κειμένων που βαθμολογήθηκαν με k ή περισσότερο, δηλαδή την κορυφή της κατανομής. Σταματάμε μόλις το τρέχον άθροισμα ξεπεράσει το $\alpha|D|$, όπου D είναι το σώμα των μη υδατογραφημένων κειμένων που χρησιμοποιούμε για βαθμονόμηση και $|D|$ το μέγεθός του. Το κατώφλι είναι τότε μία θέση πιο πάνω: το μικρότερο k που είναι ακόμη μέσα στο όριο. Τυπικά, το κατώφλι είναι:

$$\min \left\{ k : \frac{1}{|D|} \sum_{j \geq k} \text{histogram}[j] \leq \alpha \right\}$$

Ο ορισμός συνεπάγεται ότι δεν μπορούμε να πετύχουμε ακριβώς όλα τα ποσοστά ψευδώς θετικών. Πηγαινόντας προς τα πίσω ένα κελί του ιστογράμματος τη φορά, μπορεί να κάνουμε άλμα κατά πολλά

κείμενα ταυτόχρονα (αν το κελί μετρά περισσότερα από ένα κείμενα, επειδή περισσότερα από ένα κείμενα έχουν την ίδια βαθμολογία) ή να μη κινηθούμε καθόλου (αν το κελί δεν μετρά κανένα κείμενο, επειδή κανένα κείμενο δεν έχει αυτή τη βαθμολογία). Ο ορισμός παίρνει τη μακρύτερη ουρά που χωράει μέσα στο α , οπότε το ποσοστό στον οποίο καταλήγουμε μπορεί να είναι ακριβώς α , αλλά μπορεί κάλλιστα να είναι και χαμηλότερος. Επιπλέον, το α ισχύει μόνο για τα κείμενα της βαθμονόμησης· μπορεί να παρατηρήσουμε διαφορετικό ποσοστό ψευδώς θετικών σε ένα διαφορετικό σύνολο μη υδατογραφημένων κειμένων.

Η απαίτηση να μην έχει ξαναεμφανιστεί ένα παράθυρο σημαίνει ότι ο ανιχνευτής πρέπει να θυμάται τα παράθυρα που έχει δει. Υπάρχουν δύο τρόποι να γίνει αυτό. Πρώτον, μπορούμε να βάζουμε κάθε παράθυρο που συναντάμε σε ένα σύνολο, και να ελέγχουμε κάθε νέο παράθυρο αν ανήκει στο σύνολο. Αυτό είναι ακριβές, αλλά μπορεί να απαιτήσει πολλή μνήμη, επειδή το σύνολο μεγαλώνει όσο προχωράμε στο κείμενο. Ο δεύτερος τρόπος είναι να χρησιμοποιήσουμε ένα φίλτρο Bloom, που απαντά στην ίδια ερώτηση σε σταθερό αριθμό bits αλλά μπορεί να δώσει ψευδώς θετικά.

Μπορούμε να υλοποιήσουμε το φίλτρο Bloom με `nbits` ως `bytearray(nbits // 8)`. Ξεκινάμε παίρνοντας τα `k` bits που θα τεθούν, με:

```
payload = item.encode()
return [
    int.from_bytes(sha256(bytes([i]) + payload).digest(), "big") % nbits
    for i in range(k)
]
```

Το `item` είναι το παράθυρο, δοσμένο ως συμβολοσειρά, που φτιάχνεται ενώνοντας τα τεμάχια του παραθύρου με `|` ανάμεσά τους. Δεν είναι κατ' ανάγκη διακριτά όλα τα `k` bits, αλλά θα το αγνοήσουμε αυτό στον κώδικά μας—παρόλο που κάνει τα ψευδώς θετικά πιθανότερα, όπως θα δούμε στο Χειρωνακτικό Λυμένο Παράδειγμα 2 (σ. 15). Στη συνέχεια, για να θέσουμε αυτές τις θέσεις bit σε ένα `bytearray` bits χρησιμοποιούμε:

```
for bit in bit_positions:
    bits[bit >> 3] |= 1 << (bit & 7)
```

Αν το παραπάνω σας φαίνεται μυστηριώδες, ας το πάρουμε κομμάτι κομμάτι. Το `bit >> 3` ολισθαίνει δεξιά, πράγμα που είναι ισοδύναμο με ακέραια διαίρεση με το 8· το `bits[bit >> 3]` επιλέγει το συγκεκριμένο byte του `bytearray` που περιέχει τη θέση bit που θέλουμε να θέσουμε. Το `(bit & 7)` είναι ισοδύναμο με `bit % 8` για μη αρνητικούς ακέραιους. Αυτό επιλέγει μια θέση από 0 έως 7 μέσα σε αυτό το byte. Το `1 << (bit & 7)` δημιουργεί μια μάσκα με μόνο αυτό το bit στην τιμή 1, και το `|=` θέτει αυτό το bit στο επιλεγμένο byte.

Με αυτό, ο έλεγχος αν ένα στοιχείο υπάρχει στο φίλτρο Bloom γίνεται σε μία γραμμή:

```
all(bits[bit >> 3] & (1 << (bit & 7)) for bit in bit_positions)
```

Η εργασία σας είναι να γράψετε ένα πρόγραμμα σε Python που υδατογραφεί κείμενο και ανιχνεύει υδατογραφήματα, ακολουθώντας την περιγραφή της διαδικασίας που δώσαμε εδώ.

Σκαλωσιά

Σας δίνεται το αρχείο `toolkit.py`, το οποίο περιέχει τη σκαλωσιά που θα χρησιμοποιήσετε για να γράψετε την εργασία. Σας δίνεται επίσης το `synthid.py`, το οποίο περιέχει οδηγούς για τον κώδικα που θα υλοποιήσετε.

Όλα όσα είπαμε εξαρτώνται από πιθανότητες, οπότε χρειαζόμαστε έναν τρόπο να παράγουμε τυχαιότητα (ψευδοτυχαιότητα, για να είμαστε πιο ακριβείς, αλλά θα παραλείψουμε το πρόθεμα ψευδο- για συντομία). Καθώς πρόκειται για εργασία, χρειαζόμαστε η τυχαιότητα να είναι απόλυτα αναπαράξιμη, ώστε τα αποτελέσματα κάθε υποβολής να μπορούν να επαληθευτούν ανεξάρτητα. Η κλάση Randomness μας δίνει την αναγκαία λειτουργικότητα. Παρέχει τρεις μεθόδους: η randrange(bound) επιστρέφει έναν ακέραιο μεγαλύτερο ή ίσο του μηδενός και μικρότερο του bound· η random() παράγει έναν πραγματικό αριθμό στο διάστημα $0 \leq x < 1$ · οι δύο μέθοδοι χρησιμοποιούν τη randbytes(count), που επιστρέφει count τυχαία bytes.

Κανονικά, θα εφαρμόζαμε υδατογραφήματα σε κείμενο παραγόμενο από γλωσσικό μοντέλο, αλλά στο πλαίσιο της εργασίας δεν μπορούμε να έχουμε ένα πραγματικό γλωσσικό μοντέλο. Έτσι, το toolkit.py παρέχει ό,τι χρειάζεστε για να προσομοιώσετε ένα. Το λεξιλόγιο είναι οι συμβολοσειρές "w000", "w001", ..., "w999" αποθηκευμένες στην πλειάδα TOKENS μήκους VOCAB_SIZE = 1000. Το τετριμμένο αυτό μοντέλο λαμβάνει υπόψη έναν σταθερό αριθμό πιθανών τεμαχίων για κάθε θέση (48 από τα 1000 εξ ορισμού) στα οποία αποδίδει πιθανότητες.

Θα δημιουργήσετε τα αντικείμενα Randomness και γλωσσικού μοντέλου κάπου στην αρχή του προγράμματός σας:

```
rng = Randomness("generate", seed)
model = Model(MODEL_SEED, PROFILES[entropy])
```

Τα ορίσματα που περνάτε στη Randomness καθορίζουν το κείμενο: τα ίδια ορίσματα δίνουν πάντα τα ίδια bytes, οπότε χρησιμοποιήστε ακριβώς αυτά που σας υποδεικνύονται. Ειδικά για το πρώτο παράθυρο, στο οποίο δεν γίνεται Δειγματοληψία Τυρνούα, θα επιλέγετε τεμάχια από την πλειάδα TOKENS με TOKENS[rng.randrange(VOCAB_SIZE)]. Από εκεί και πέρα, κάθε φορά που καλείτε τη μέθοδο model.distribution(window), θα παίρνετε ένα στιγμιότυπο της κλάσης Distribution. Το στιγμιότυπο περιέχει τους δείκτες indices των τεμαχίων που λαμβάνονται υπόψη και τις αντίστοιχες πιθανότητές τους, probs. Αυτό σημαίνει ότι για τον δείκτη i, το ίδιο το τεμάχιο είναι το TOKENS[indices[i]]. Κάθε τεμάχιο του οποίου ο δείκτης δεν βρίσκεται στο indices έχει πιθανότητα μηδέν.

Για να κληρώσετε ένα τεμάχιο, χρησιμοποιήστε την choose(probs, rng.random()). Διατρέχει το αθροιστικό άθροισμα των πιθανοτήτων και επιστρέφει μια θέση στη λίστα που της περάσατε. Αν επιστρέψει i, το τεμάχιο είναι το TOKENS[indices[i]]. Κάθε τεμάχιο μετά το πρώτο παράθυρο κληρώνεται με αυτήν την κλήση. Υπάρχουν τρεις διαφορετικές περιπτώσεις για τη λίστα probs που δίνεται στην κλήση. Όταν δεν υδατογραφείτε, περνάτε τις πιθανότητες που παρήγαγε το μοντέλο. Όταν υδατογραφείτε, περνάτε την τελική λίστα πιθανοτήτων στον διαστρωματωμένο δειγματολήπτη· ή πάλι τις ίδιες τις πιθανότητες του μοντέλου, μία φορά για κάθε υποψήφιο που κληρώνεται, όταν διεξάγετε αγώνες νοκ-άουτ.

Για να διαβάσετε αρχεία μέσα στο πρόγραμμά σας θα χρησιμοποιήσετε τη read_documents(path), η οποία επιστρέφει μία λίστα τεμαχίων ανά γραμμή του αρχείου, παραλείποντας τις κενές γραμμές.

Το κείμενο παράγεται στοχαστικά, αλλά μπορούμε να ελέγξουμε το επίπεδο τυχαιότητας που θα χρησιμοποιήσει το μοντέλο. Συγκεκριμένα, μπορούμε να ρυθμίσουμε τον αριθμό των τεμαχίων που θα λαμβάνονται υπόψη σε κάθε θέση και την κατανομή πιθανότητας που αποδίδεται σε αυτά. Αυτό περιγράφεται από την κλάση EntropyProfile. Ο αριθμός των τεμαχίων που λαμβάνονται υπόψη δίνεται από το support, ενώ η κατανομή πιθανότητας ελέγχεται από το alpha. Έτσι, στο r-οστό στοιχείο της κατανομής αποδίδεται πιθανότητα ανάλογη του $1 / r^{**alpha}$ · τα βάρη κλιμακώνονται στη συνέχεια ώστε να αθροίζουν στο ένα. Στην εργασία θα χρησιμοποιήσετε τα δύο προφίλ "high" και "low", που ορίζονται στο PROFILES.

Τρεις συναρτήσεις σας δίνονται έτοιμες. Η g_value(key, window, layer, token) επιστρέφει την

τιμή g . Η `context_item(window)` ενώνει ένα παράθυρο στη συμβολοσειρά που χρησιμοποιείται ως κλειδί. Και για να πάρετε τις k θέσεις bit ενός στοιχείου στο φίλτρο Bloom, χρησιμοποιήστε τη `bloom_hash_positions(item, nbits, k)`.

Στο τέλος του `toolkit.py` θα βρείτε τον αναλυτή ορισμάτων της γραμμής εντολών, οπότε δεν χρειάζεται να τον φτιάξετε μόνοι σας. Δεν θα τον καλέσετε απευθείας: η `run(generate, detect)` αναλύει τα ορίσματα, καλεί όποια από τις δύο συναρτήσεις σας ζητά ο τρόπος λειτουργίας, και τυπώνει το αποτέλεσμα. Η τελευταία γραμμή του `synthid.py` το κάνει ήδη αυτό. Γι' αυτό οι δικές σας `generate` και `detect` επιστρέφουν το αντικείμενο αντί να το τυπώνουν. Αν το τυπώνετε εσείς, θα βγαίνανε δύο αντικείμενα JSON στην πρότυπη έξοδο, και το αποτέλεσμα δεν θα μπορούσε να βαθμολογηθεί.

Χρήση

Το πρόγραμμά σας, `synthid.py`, θα καλείται με δύο τρόπους. Ο πρώτος παράγει υδατογραφημένο κείμενο. Ο δεύτερος το ανιχνεύει.

```
python synthid.py --generate --key KEY --seed 42 --length 400
python synthid.py --detect --key KEY --calibration cal.txt --documents docs.txt
```

Και στις δύο περιπτώσεις το πρόγραμμα θα τυπώνει ακριβώς ένα αντικείμενο JSON στην πρότυπη έξοδο, και τίποτε άλλο. Η πλήρης μορφή δίνεται από το σχήμα JSON στο `output.schema.json`. Τρέξτε:

```
python validate.py my_output.json
```

σε ό,τι παράγετε προτού βασιστείτε σε αυτό στην διάρκεια της επίλυσης της εργασίας.

Τρόπος 1: Παραγωγή Υδατογραφημένου Κειμένου

Για να υδατογραφήσετε κείμενο θα καλείτε το πρόγραμμα ως εξής:

```
python synthid.py --generate --key KEY --seed 42 --length 400 \
    [--layers 30] [--window 4] \
    [--entropy high|low] [--sampler layered|knockout] [--no-watermark]
```

παράμετρος	τι κάνει
<code>--seed</code>	καθορίζει την τυχαιότητα
<code>--length</code>	αριθμός τεμαχίων προς παραγωγή
<code>--layers</code>	επίπεδα τουρνουά $m \cdot 30$ εξ ορισμού
<code>--window</code>	μήκος συμφραζομένων $H \cdot 4$ εξ ορισμού
<code>--entropy</code>	η ελευθερία του τετριμμένου μοντέλου
<code>--sampler</code>	ποιον από τους δύο δειγματολήπτες σας να χρησιμοποιήσει
<code>--no-watermark</code>	κλήρωση κατευθείαν από το μοντέλο, χωρίς τουρνουά

Δύο εκτελέσεις με τον ίδιο σπόρο και τις ίδιες σημαίες πρέπει να παράγουν πάντοτε *ακριβώς* τα ίδια τεμάχια, σε οποιοδήποτε μηχάνημα. Θα πρέπει να χρησιμοποιήσετε αποκλειστικά τη Randomness για οποιοδήποτε στοχαστικό γεγονός στο πρόγραμμά σας.

Όπως είδαμε, υπάρχουν δύο τρόποι να υλοποιηθεί η Δειγματοληψία Τουρνουά, και θα υλοποιήσετε και τους δύο στο πρόγραμμά σας, με δύο ξεχωριστές συναρτήσεις.

Η `sample_knockout(p, g, m, rng)` θα επιστρέφει ένα τεμάχιο, διεξάγοντας ένα τουρνουά νοκ-άουτ με m στάδια, όπου κάθε αγώνας είναι ένα στιγμότυπο του αλγορίθμου ενός επιπέδου. Οι πιθανότητες του μοντέλου για όλα τα τεμάχια δίνονται από το p . Θα κληρώνετε 2^m υποψηφίους: θα τους βά-ζετε σε ζεύγη με τη σειρά που κληρώθηκαν, θα αφήνετε την υψηλότερη τιμή g να κερδίζει κάθε αγώνα,

και θα επαναλαμβάνετε για m γύρους ώσπου να μείνει ένας. Θα κληρώνετε τον i -οστό υποψήφιο με `choose(p, rng.random())`. Σε ισοπαλία με δύο ίσες τιμές g κερδίζει ο αριστερός υποψήφιος.

Η `sample_layered(p, g, m)` θα επιστρέφει μια κατανομή: αυτήν από την οποία κληρώνει το τουρνουά, με το ίδιο μήκος όπως το p , που πάλι είναι η αρχική κατανομή, εφαρμόζοντας τη διαστρωματωμένη ενημέρωση των πιθανοτήτων μία φορά ανά επίπεδο.

Η δεύτερη συνάρτηση επιστρέφει ολόκληρη κατανομή, η πρώτη ένα μόνο τεμάχιο: η `sample_layered` δίνει τη λίστα πιθανοτήτων από την οποία κληρώνει η `sample_knockout` ώστε να επιστρέψει το νικητήριο τεμάχιο. Αν τρέξετε το νοκ-άουτ πολλές φορές, και μετρήσετε πόσο συχνά εμφανίζεται κάθε τεμάχιο, θα πρέπει να ανακτήσετε αυτό που επιστρέφει η διαστρωματωμένη εκδοχή σε ένα πέρασμα.

Εξ ορισμού, ο τρόπος `--generate` πρέπει να χρησιμοποιεί τη `sample_layered`. Ο προεπιλεγμένος αριθμός επιπέδων είναι 30, οπότε ένα νοκ-άουτ θα χρειαζόταν 2^{30} , παραπάνω από ένα δισεκατομμύριο, κληρώσεις υποψηφίων για ένα μόνο τεμάχιο. Δοκιμάστε `--sampler knockout --layers 30` μία φορά και περιμένετε να τελειώσει—ή μάλλον καλύτερα μην το κάνετε.

Μετά από κάθε επίπεδο, ελέγξτε ότι οι πιθανότητες αθροίζουν ακόμη στο 1, προσαρμόζοντάς την ανοχή στη σύγκριση με το m . Η ενημέρωση υψώνει το σύνολο στο τετράγωνο: τα δύο σύνολα A και B συνεισφέρουν q_A^2 και $2q_Aq_B + q_B^2$, που αθροίζουν σε $(q_A + q_B)^2$. Έτσι ένα σύνολο $1 + \varepsilon$ καταλήγει σε $1 + 2\varepsilon + \varepsilon^2$, και το σφάλμα στρογγυλοποίησης διπλασιάζεται σε κάθε επίπεδο: ένα αρχικό σφάλμα 10^{-16} έχει μεγαλώσει σε περίπου 10^{-7} ως το επίπεδο 30. Για ανοχή, χρησιμοποιήστε:

```
tolerance = max(1e-9, 2**m * 1e-15)
```

Μη χρησιμοποιήσετε `sum(p) == 1`, που αποτυγχάνει ακόμη και σε σωστό κώδικα με $m = 2$: το Χειρωνακτικά Λυμένο Παράδειγμα 1 (σ. 14) καταλήγει στο άθροισμα 1,0000000000000002.

Τρόπος 2: Εντοπισμός Υδατογραφήματος

Για να ανιχνεύσετε υδατογραφημένο κείμενο, καλέστε το πρόγραμμα με:

```
python synthid.py --detect --key KEY --calibration cal.txt --documents docs.txt \
    [--alpha 0.01] [--layers 30] [--window 4] \
    [--bloom-bits 512] [--bloom-hashes 3]
```

Το πρόγραμμα βαθμολογεί κάθε κείμενο του σώματος βαθμονόμησης, υπολογίζει το κατώφλι για το δοσμένο `--alpha`, και μετά βαθμολογεί και κρίνει κάθε κείμενο του `--documents`. Η έξοδος περιέχει το κατώφλι, μία εγγραφή για κάθε κρινόμενο κείμενο, και σε πόσα εντοπίστηκε υδατογράφημα.

παράμετρος	τι κάνει
<code>--calibration</code>	το μη υδατογραφημένο αρχείο πάνω στο οποίο ορίζεται το κατώφλι
<code>--documents</code>	το αρχείο με τα κείμενα προς κρίση
<code>--alpha</code>	το ποσοστό ψευδώς θετικών για βαθμονόμηση· 0,01 εξ ορισμού
<code>--layers</code>	επίπεδα τουρνουά m · 30 εξ ορισμού
<code>--window</code>	μήκος συμφραζομένων H · 4 εξ ορισμού
<code>--bloom-bits</code>	μέγεθος του φίλτρου Bloom, σε bits
<code>--bloom-hashes</code>	πόσες συναρτήσεις κατακερματισμού χρησιμοποιεί· 3 εξ ορισμού

Οι δύο τελευταίες παράμετροι καθορίζουν πώς ο ανιχνευτής θυμάται τα παράθυρα που έχει δει. Χωρίς `--bloom-bits` χρησιμοποιεί ένα `set`, και το αντικείμενο bloom παραλείπεται εντελώς από την έξοδο. Με την παράμετρο, ο ανιχνευτής χρησιμοποιεί ένα φίλτρο Bloom με τόσα bits όσα δίνει η παράμετρος και `--bloom-hashes` συναρτήσεις κατακερματισμού αντί για `set`. Το `--bloom-bits` είναι το nbits της `bloom_scored_positions` και το `--bloom-hashes` το k της. Κάθε βαθμολογημένο κείμενο αναφέρει τον

αριθμό των ψευδώς θετικών στο `false_skips`. Αυτά είναι πόσες θέσεις παρέκαμψε το φίλτρο τις οποίες το ακριβές σύνολο θα είχε βαθμολογήσει.

Θα βαθμονομείτε πάνω σε μη υδατογραφημένο κείμενο του ίδιου μήκους με το κείμενο που κρίνετε. Όπως εξηγήσαμε, ένα κατώφλι από κείμενα 200 τεμαχίων δεν έχει νόημα για κείμενα 25 τεμαχίων. Βάζετε κατώφλι στον αριθμητή, και ο αριθμητής μεγαλώνει με το κείμενο: ένα κείμενο 25 τεμαχίων βαθμολογείται περίπου στο 315 ενώ ένα κείμενο 200 τεμαχίων περίπου στο 2941, οπότε ένα κατώφλι 3031 που πάρθηκε στα 200 τεμάχια δεν θα σημάδευε τίποτε στα 25 τεμάχια. Το όνομα κάθε αρχείου βαθμονόμησης περιέχει το μήκος των κειμένων που περιέχει, ώστε να ξέρετε να ταιριάζετε τα κείμενα προς έλεγχο με τα σωστά κείμενα βαθμονόμησης.

Για τη βαθμολόγηση θα υλοποιήσετε τα ακόλουθα:

H `scored_positions(tokens, h)` θα επιστρέφει τις βαθμολογήσιμες θέσεις για το κείμενο που αποτελείται από τα `tokens`. Αυτές είναι όσες έχουν πλήρες παράθυρο `h` προηγούμενων τεμαχίων και των οποίων τα συμφραζόμενα δεν έχουν εμφανιστεί νωρίτερα στο κείμενο. Κρατήστε τα συμφραζόμενα που έχετε δει σε ένα `set` της Python· κάθε συμφραζόμενο αποτελείται από `h` τεμάχια συμφραζόμενων ενωμένα με "|", χωρίς το τεμάχιο που θα βαθμολογηθεί.

H `score_numerator(tokens, key, h, m)` θα επιστρέφει τον αριθμητή του τύπου βαθμολόγησης, για το κείμενο που αποτελείται από τα `tokens` χρησιμοποιώντας το `key` για τον υπολογισμό των τιμών `g`. Το μήκος του παραθύρου τεμαχίων είναι `h`, και το `m` είναι ο αριθμός των επιπέδων.

H `mean_score(tokens, key, h, m)` θα επιστρέφει τη μέση βαθμολογία του κειμένου. Τα ορίσματά της είναι τα ίδια με αυτά της `score_numerator`.

H `BloomFilter(nbits, k)` είναι το ίδιο το φίλτρο, μια κλάση με δύο μεθόδους. Η πρώτη, η `add(item)`, θέτει τα `bits` ενός στοιχείου. Η δεύτερη, η `contains(item)`, απαντά αν είναι όλα αυτά τα `bits` έχουν τιμή 1.

H `bloom_scored_positions(tokens, h, nbits, k)` θα επιστρέφει τις βαθμολογήσιμες θέσεις για το κείμενο που αποτελείται από τα `tokens` χρησιμοποιώντας φίλτρο Bloom αντί για `set` της Python. Το φίλτρο Bloom θα υλοποιηθεί σε μια κλάση `BloomFilter` και θα χρησιμοποιεί `nbits` bits, θέτοντας το πολύ `k` από αυτά για κάθε παράθυρο συμφραζόμενων, επειδή δύο συναρτήσεις κατακερματισμού μπορούν να συγκρουστούν στο ίδιο bit. Κάθε συμφραζόμενο αναπαρίσταται όπως στη `scored_positions(tokens, h)`.

Για την ανίχνευση, θα υλοποιήσετε:

Την **`histogram(numerators, length, m)`**, η οποία θα καταμετρά τους δοσμένους `numerators` σε έναν πίνακα: το `hist[k]` μετρά τα κείμενα των οποίων ο αριθμητής ήταν ακριβώς `k`. Τα κείμενα έχουν `length` τεμάχια το καθένα και βαθμολογήθηκαν πάνω σε `m` επίπεδα.

Την **`threshold(hist, n_docs, alpha)`**, η οποία θα επιστρέφει το κατώφλι: τον μικρότερο αριθμητή του οποίου η άνω ουρά στο `hist` περιέχει το πολύ ένα ποσοστό `alpha` από τα `n_docs` κείμενα βαθμονόμησης που μπήκαν σε αυτό. Ο ανιχνευτής αποφαινεται ότι ένα κείμενο έχει υδατογράφημα όταν ο αριθμητής φτάνει αυτό το κατώφλι.

Τα Διαθέσιμα Κείμενα

Θα δουλέψετε με το σύνολο κειμένων στον κατάλογο `corpus/`. Κάθε αρχείο περιέχει ένα κείμενο ανά γραμμή, με τα τεμάχια χωρισμένα με κενά. Διαβάστε ένα με τη `read_documents(path)` από το `toolkit.py`, που επιστρέφει μια λίστα κειμένων, κάθε ένα μία λίστα τεμαχίων. Μην ανοίγετε τα αρχεία

μόνοι σας: η `open` είναι μία από τις ενσωματωμένες συναρτήσεις που δεν επιτρέπεται να χρησιμοποιήσει το πρόγραμμά σας.

Τα κείμενα δημιουργήθηκαν με το κλειδί υδατογράφησης:

`rwa-synthid-2026`

οπότε αυτό είναι που περνάτε ως `--key` όποτε ανιχνεύετε μέσα σε αυτό. Το κλειδί δεν είναι μυστικό (πρόκειται για εργασία, όχι για πραγματικό σύστημα!), αλλά πρέπει οπωσδήποτε να είναι το σωστό: ένας ανιχνευτής που τρέχει με οποιοδήποτε άλλο κλειδί δεν βλέπει παρά θόρυβο, και δεν ανιχνεύει απολύτως τίποτα.

αρχείο	τι είναι
<code>calibration_T025.txt ... T200.txt</code>	1000 μη υδατογραφημένα κείμενα σε κάθε μήκος
<code>unwatermarked_T025.txt ... T200.txt</code>	200 ακόμη μη υδατογραφημένα κείμενα, κρατημένα στην άκρη
<code>watermarked_T025.txt ... T200.txt</code>	200 υδατογραφημένα κείμενα σε κάθε μήκος
<code>low_entropy_T200.txt</code>	200 υδατογραφημένα κείμενα από μοντέλο χωρίς ελευθερία
<code>repeats_T200.txt</code>	200 υδατογραφημένα κείμενα που λένε τα ίδια δύο φορές
<code>mystery.txt</code>	10 κείμενα· κάποια είναι υδατογραφημένα, δεν ξέρετε ποια

Τα μήκη είναι 25, 50, 100 και 200 τεμάχια. Τα αρχεία βαθμονόμησης είναι αυτά που περνάτε στο `--calibration`. Όλα τα υπόλοιπα αρχεία είναι για ανίχνευση.

Σε κάθε αρχείο εκτός από το `repeats_T200.txt` δεν έχουμε επαναλήψεις παραθύρων. Κάθε κείμενο σε αυτό το αρχείο, όμως, επαναλαμβάνει ένα απόσπασμα του εαυτού του, και το καθένα έχει επίσης τέσσερα παράθυρα που ξαναεμφανίζονται αργότερα με διαφορετικό τεμάχιο μετά από αυτά. Τρέξτε τον ανιχνευτή σας πάνω σε αυτό και το `scorable` θα πρέπει να βγει μεταξύ περίπου 150 και 170, αντί για 196.

Έξοδος του Ανιχνευτή

Το πεδίο `flagged` στην έξοδο JSON του προγράμματός σας θα μετρά τα κείμενα που ο ανιχνευτής σας χαρακτήρισε υδατογραφημένα. Η ακριβής σημασία του εξαρτάται από τα κείμενα που διαβάζει το πρόγραμμά σας.

Στα `unwatermarked_T*.txt`, το ποσοστό που έχει ανιχνευτεί είναι το ποσοστό ψευδώς θετικών και θα πρέπει να είναι κοντά στο α . Στα `watermarked_T*.txt`, το ποσοστό που έχει ανιχνευτεί είναι το ποσοστό αληθώς θετικών για εκείνο το α , δηλαδή το $\text{TPR@FPR} = \alpha$. Καθώς το μήκος των κειμένων αυξάνεται, το ποσοστό αληθώς θετικών μεγαλώνει: στα 25 τεμάχια το υλικό που έχουμε στη διάθεσή μας είναι περιορισμένο, αλλά στα 200 τεμάχια δεν υπάρχει περίπτωση να μας διαφύγει το υδατογράφημα.

Το `mystery.txt` περιέχει 10 κείμενα των 200 τεμαχίων. Τα μισά είναι υδατογραφημένα και τα μισά όχι, οπότε το πλήθος `flagged` του δεν είναι ποσοστό, είναι η καταμέτρηση δέκα ανιχνεύσεων. Για κείμενα για τα οποία δεν ξέρουμε αν είναι υδατογραφημένα ή όχι, αυτό που θέλουμε είναι τα πεδία `flagged` για το εκάστοτε κείμενο, όχι το σύνολο. Μπορείτε να αξιολογήσετε το `mystery.txt` με `--detect --alpha 0.01`, βαθμονομώντας με το `calibration_T200.txt`, και μετά να ελέγξετε το πεδίο `flagged` κάθε εγγραφής.

Στο `low_entropy_T200.txt`, το ποσοστό αληθώς θετικών καταρρέει σχεδόν στο μηδέν. Το σώμα χαμηλής εντροπίας γράφτηκε από τον χρησιμοποιώντας ένα μοντέλο με σχεδόν καμία ελευθερία, δηλαδή χαμηλή εντροπία: σε κάθε βήμα ένα τεμάχιο καταλαμβάνει περίπου το 99,6% της πιθανότητας. Αυτή είναι η ουσία και το όριο όλης της προσέγγισης: το υδατογράφημα υπάρχει εφόσον το μοντέλο έχει ελεύθερες επιλογές για την επιλογή του επόμενου τεμαχίου. Το τουρνουά μπορεί να προωθήσει ένα τεμάχιο με $g = 1$ μόνο αν υπάρχει κάποιο άλλο εναλλακτικό τεμάχιο. Όταν ένα τεμάχιο καταλαμβάνει όλη την πιθανότητα, το τουρνουά επιστρέφει εκείνο το τεμάχιο όποια και αν είναι η τιμή g του, οπότε

δεν υπάρχει τίποτε να ανιχνευθεί. Μπορείτε να το δείτε μόνοι σας παράγοντας τα δικό σας κείμενα με `--generate --entropy low`. Μην εκπλαγείτε αν δεν δείτε επαναλαμβανόμενα αποσπάσματα σε κείμενα χαμηλής εντροπίας. Τα κείμενα χαμηλής εντροπίας χρησιμοποιούν περίπου τόσα διαφορετικά τεμάχια όσα και τα άλλα κείμενα, ώστε κάθε θέση εξακολουθεί να περνά τον έλεγχο ότι δεν έχουμε ξαναδεί τα συμφραζόμενά της. Παίρνουμε χαμηλή εντροπία εξαναγκάζοντας την πιθανότητα του επόμενου τεμαχίου, όχι απλώς επαναλαμβάνοντάς το.

Βαθμολόγηση

Η βαθμολόγηση θα βασιστεί στην αξιολόγηση και των δύο τρόπων λειτουργίας του προγράμματός σας, αναλυμένων σε τρία επιμέρους τμήματα, καθώς και στον έλεγχο ότι η έξοδος του προγράμματός σας συμμορφώνεται με το προδιαγεγραμμένο σχήμα JSON. Το Τμήμα Α αφορά την παραγωγή υδατογραφημένων κειμένων με τους δύο δειγματολήπτες. Το Τμήμα Β είναι η βαθμολόγηση κειμένων και η χρήση του φίλτρου Bloom. Το Τμήμα Γ είναι η βαθμονόμηση μέσω της δημιουργίας του ιστογράμματος των βαθμολογιών των κειμένων και η ανίχνευση υδατογραφημένων κειμένων. Κάθε τμήμα αναλύεται περαιτέρω σε δύο μέρη, με συγκεκριμένες μονάδες, που φαίνονται στον πίνακα που ακολουθεί:

	μονάδες
Έλεγχος ότι η έξοδος συμφωνεί με το σχήμα JSON	5
A.1: παραγωγή με τον δειγματολήπτη νοκ-άουτ, και συμφωνία με τον διαστρωματωμένο	10
A.2: παραγωγή με τον διαστρωματωμένο δειγματολήπτη	25
B.1: βαθμολόγηση κειμένων	20
B.2: βαθμολόγηση κειμένων με το φίλτρο Bloom	15
Γ.1: βαθμονόμηση	15
Γ.2: ανίχνευση	10
σύνολο	100

Οι μονάδες θα προκύψουν από την εκτέλεση του προγράμματός σας και τη σύγκριση της εξόδου του με την έξοδο του προγράμματος αναφοράς, σε κλήσεις που δεν έχετε δει, με διαφορετικούς σπόρους, διαφορετικά μήκη, διαφορετικές τιμές των `--layers` και `--window`, και διαφορετικό σύνολο κειμένων. Δίνουμε Παραδείγματα Κλήσεων (από τη σ. 18 και μετά) ώστε να μπορείτε να ελέγχετε μόνοι σας. Το σώμα κειμένων στο οποίο βαθμολογείστε κατασκευάζεται ξεχωριστά από αυτά που σας δίνονται, με διαφορετικά μήκη και με διαφορετικό κλειδί.

Θυμηθείτε ότι το πρόγραμμά σας πρέπει να τυπώνει ακριβώς ένα αντικείμενο JSON και τίποτε άλλο. Διαφορετικά δεν θα είναι δυνατόν να διαβαστεί, με αποτέλεσμα μηδέν μονάδες. Οι 5 μονάδες αξιολογούν ότι η έξοδος συμφωνεί με το σχήμα, επειδή το πρόγραμμά σας μπορεί να βγάζει JSON, αλλά όχι το JSON που προδιαγράφουμε.

Χειρωνακτικά Λυμένο Παράδειγμα 1

Δειγματοληψία Τουρνουά με $|V| = 4$, $m = 2$ και τεμάχια A, B, C, D με $p = [0.4, 0.3, 0.2, 0.1]$.

Επίπεδο 1, $g = [1, 0, 1, 0]$:

Zero set {B, D}: $q_A = 0.3 + 0.1 = 0.4$

One set {A, C}: $q_B = 0.4 + 0.2 = 0.6$

Multiplier for the one set: $2 \cdot q_A + q_B = 0.8 + 0.6 = 1.4$

A: $0.4 \cdot 1.4 = 0.56$

B: $0.3 \cdot 0.4 = 0.12$

C: $0.2 \cdot 1.4 = 0.28$

D: $0.1 \cdot 0.4 = 0.04$


```
p = [0.56, 0.12, 0.28, 0.04]      sum = 1.00
```

Επίπεδο 2, $g = [0, 0, 1, 1]$:

Zero set {A, B}: $q_A = 0.56 + 0.12 = 0.68$

One set {C, D}: $q_B = 0.28 + 0.04 = 0.32$

Multiplier: $2 \cdot q_A + q_B = 1.36 + 0.32 = 1.68$

A: $0.56 \cdot 0.68 = 0.3808$ B: $0.12 \cdot 0.68 = 0.0816$

C: $0.28 \cdot 1.68 = 0.4704$ D: $0.04 \cdot 1.68 = 0.0672$

```
p = [0.3808, 0.0816, 0.4704, 0.0672]      sum = 1.0000
```

Το C ξεκίνησε στο 0,2 και καταλήγει ψηλότερα από όλα στο 0,4704: έχει $g = 1$ και στα δύο επίπεδα. Το B έχει $g = 0$ και στα δύο και πέφτει από 0,3 σε 0,0816. Το A και το D κερδίζουν ένα επίπεδο το καθένα και χάνουν ένα.

Η δική σας `sample_knockout` πρέπει να αναπαράγει αυτούς τους τέσσερις αριθμούς ως συχνότητες δειγματοληψίας. Σε 10^6 κληρώσεις αναμένετε περίπου $380.800 / 81.600 / 470.400 / 67.200$, με διακύμανση κάποιων εκατοντάδων στο καθένα. Ελέγχετε ότι `abs(sum(p) - 1) < 1e-9` μετά από κάθε επίπεδο, και όχι `sum(p) == 1`. Ο πολλαπλασιασμός πραγματικών αριθμών δεν διατηρεί το άθροισμα ακριβώς. Σε αυτό το παράδειγμα, το άθροισμα μετά το επίπεδο 2 είναι 1.0000000000000002 , οπότε ο έλεγχος ισότητας αποτυγχάνει ακόμη και σε σωστό κώδικα. Αν όμως αποτύχει ο έλεγχος με την ανοχή που έχουμε αναφέρει παραπάνω, τότε σχεδόν σίγουρα έχετε εφαρμόσει μόνο τον ένα από τους δύο πολλαπλασιαστές.

Χειρωνακτικά Λυμένο Παράδειγμα 2

Φίλτρο Bloom με $nbits = 16$, $k = 3$. Για την κωδικοποίηση των στοιχείων χρησιμοποιούμε $h = 2$ τεμάχια συμφραζομένων ενωμένα με "|". Η πρόταση "the cat sat on the mat" δίνει τα συμφραζόμενα "the|cat", "cat|sat", "sat|on", "on|the", "the|mat". Ο κατακερματισμός i του στοιχείου x είναι:

```
int.from_bytes(sha256(bytes([i]) + x.encode()).digest(), "big") % nbits
```

Ξεκινάμε με όλα τα 16 bits μηδέν και προσθέτουμε τα δύο πρώτα συμφραζόμενα:

```
"the|cat" -> bits 7, 3, 6
```

```
"cat|sat" -> bits 15, 9, 15
```

```
bitarray: 0 0 0 1 0 0 1 1 0 1 0 0 0 0 0 1
           indices set to 1: {3, 6, 7, 9, 15}
```

Έχουμε δύο στοιχεία και έξι υπολογισμούς κατακερματισμού, αλλά μόνο πέντε διαφορετικά bits, επειδή το "cat|sat" συγκρούεται με τον εαυτό του: δύο από τους τρεις κατακερματισμούς του συγκρούονται στο bit 15, οπότε θέτει δύο bits αντί για τρία. Οι συναρτήσεις κατακερματισμού μπορούν να συγκρουστούν μεταξύ τους, πράγμα που κάνει τα ψευδώς θετικά πιθανότερα από ό,τι θα υπέβαλλαν k διαφορετικά bits.

Ας ρωτήσουμε τώρα για δύο συμφραζόμενα που δεν προστέθηκαν ποτέ:

```
"sat|on" -> bits 2, 13, 5
           bit 2 is zero, so NOT PRESENT (correct)
```

```
"mat|on" -> bits 15, 7, 3
```

all set, so PRESENT (wrong -- a false positive)

Το "mat|on" αποτελείται από γνωστές λέξεις αλλά δεν εμφανίστηκε ποτέ ως συμφραζόμενο· και τα τρία bits του έτυχε να έχουν τεθεί από τα δύο στοιχεία που προστέθηκαν.

Στο B.2 ένα ψευδώς θετικό σημαίνει ότι ο ανιχνευτής πιστεύει ότι έχει βαθμολογήσει ήδη μια θέση και την παρακάμπτει. Τότε θα αγνοήσουμε όλες τις m τιμές g για το τεμάχιο, απορρίπτοντας αποδεικτικά στοιχεία για την ανίχνευση. Το φίλτρο Bloom δεν θα παράξει ποτέ ψευδώς αρνητικά, που θα είχε ως συνέπεια να προσθέσουμε στον αριθμητή τιμές g που θα έπρεπε να αγνοήσουμε. Επομένως, το φίλτρο δεν θα πλαστογραφήσει ποτέ αποδεικτικά στοιχεία για την ανίχνευση.

Το φίλτρο των 16 bits εδώ είναι τετριμμένο, μόνο για επίδειξη. Στο B.2 θα βαθμολογηθείτε με nbits στο {512, 2048, 8192}.

Έλεγχος της Εργασίας Σας

Για να ελέγξετε την εργασία σας χρησιμοποιήστε:

```
python test_public.py          # or: pytest test_public.py
```

Αυτοί είναι οι ίδιοι έλεγχοι που κάνει ο βαθμολογητής, στα Χειρωνακτικά Λυμένα Παραδείγματα και σε άλλες εισόδους που κατασκευάζονται επί τούτου. Το να περάσετε όλους τους ελέγχους δεν εγγυάται πλήρη βαθμό, επειδή ο βαθμολογητής τρέχει το πρόγραμμά σας σε κλήσεις που δεν έχετε δει—αλλά αν το πρόγραμμά σας αποτυγχάνει εδώ, σίγουρα θα αποτύχει και στον βαθμολογητή.

Θα πρέπει επίσης να χρησιμοποιήσετε τα Παραδείγματα Κλήσεων που ξεκινούν στη σ. 18. Το καθένα ονοματίζει το αρχείο μέσα στο examples/ που κρατά την έξοδο που πρέπει να παραγάγει, οπότε μπορείτε να χρησιμοποιήσετε το diff για να σας πει αν το πρόγραμμά σας συμφωνεί με το πρόγραμμα αναφοράς σε μια πραγματική κλήση, από άκρο σε άκρο.

Υπάρχουν δύο ακόμη έλεγχοι που μπορείτε να τρέξετε απευθείας. Πρώτον, ελέγξτε ότι τα κείμενα μπορούν να παραχθούν ντετερμινιστικά:

```
python synthid.py --generate --key K --seed 7 --length 100 > a.json
python synthid.py --generate --key K --seed 7 --length 100 > b.json
diff a.json b.json          # must be empty
```

Δεύτερον, ελέγξτε ότι το υδατογράφημα πραγματικά υπάρχει. Παραγάγετε ένα κείμενο, κρίνετέ το, και κρίνετε ένα γραμμένο χωρίς το υδατογράφημα:

```
python synthid.py --generate --key K --seed 7 --length 200 \
| python -c "import json,sys; print(' '.join(json.load(sys.stdin)['tokens']))" >
↪ mine.txt
python synthid.py --detect --key K --calibration corpus/calibration_T200.txt \
--documents mine.txt --alpha 0.01
```

Η βαθμολογία θα πρέπει να είναι πολύ πάνω από 0,5, και το flagged θα πρέπει να είναι 1. Επαναλάβετε με --no-watermark και θα πρέπει να είναι 0. Μπορείτε να χρησιμοποιήσετε οποιοδήποτε κλειδί, όχι κατ' ανάγκη αυτό που χρησιμοποιήθηκε για την παραγωγή των κειμένων βαθμονόμησης. Αυτό που πρέπει να ταιριάζει είναι το κλειδί που χρησιμοποιείτε για την παραγωγή και το κλειδί που χρησιμοποιείτε για την ανίχνευση.

Προϋποθέσεις Υποβολής

Κάθε φοιτητής θα εργαστεί σε αποθετήριο στο GitHub. Για να αξιολογηθεί μια εργασία θα πρέπει να πληροί τις παρακάτω προϋποθέσεις:

- Για την υποβολή της εργασίας θα χρησιμοποιηθεί το ιδιωτικό αποθετήριο του φοιτητή που δημιουργήθηκε για τις ανάγκες του μαθήματος και του έχει αποδοθεί. Το αποθετήριο αυτό έχει όνομα του τύπου `username-algo-assignments`, όπου `username` είναι το όνομα του φοιτητή στο GitHub. Για παράδειγμα, το σχετικό αποθετήριο του διδάσκοντα (με GitHub username `louridas`) θα ονομαζόταν `louridas-algo-assignments` και θα ήταν προσβάσιμο στο `https://github.com/dmst-algorithms-course/louridas-algo-assignments`. *Τυχόν άλλα αποθετήρια απλώς θα αγνοηθούν.*
 - Μέσα στο αποθετήριο αυτό θα πρέπει να δημιουργηθεί ένας κατάλογος `assignment-2026-4`.
 - Μέσα στον παραπάνω κατάλογο το πρόγραμμα θα πρέπει να αποθηκευτεί με το όνομα `synthid.py`.
 - Δεν επιτρέπεται η χρήση έτοιμων βιβλιοθηκών ή έτοιμων λύσεων, ή τμημάτων αυτών, εκτός αν αναφέρεται ρητά ότι επιτρέπεται.
 - Επιτρέπεται η χρήση δομών δεδομένων της πρότυπης βιβλιοθήκης της Python, όπως `στοίβες` (λίστες), `λεξικά`, `σύνολα`, κ.λπ.
 - Επιτρέπεται η χρήση των παρακάτω βιβλιοθηκών, και καμίας άλλης: `argparse`, `array`, `bisect`, `collections`, `collections.abc`, `dataclasses`, `enum`, `functools`, `hashlib`, `heapq`, `itertools`, `json`, `math`, `operator`, `random`, `string`, `sys`, `typing`. Θα πρέπει φυσικά να εισαγάγετε και το `toolkit`, που σας δίνουμε εμείς.
- Οτιδήποτε άλλο, συμπεριλαμβανομένου του `numpy`, θα μηδενίσει τις ενότητες που βαθμολογούνται με εκτέλεση του κώδικά σας. Ο έλεγχος γίνεται προτού εισαχθεί το αρχείο σας, οπότε η παράνομη εισαγωγή δεν εκτελείται καν. Το ίδιο ισχύει για τα `eval`, `exec`, `open`, `__import__`, και για την προσπέλαση `dunder` γνωρισμάτων.
- Το πρόγραμμα θα πρέπει να είναι γραμμένο σε Python 3.14.
 - Η εργασία είναι αποκλειστικά ατομική. Δεν επιτρέπεται συνεργασία μεταξύ φοιτητών, με ποινή τον μηδενισμό. Επιπλέον, το πρόγραμμα που υποβάλλεται δεν μπορεί να είναι αποτέλεσμα συστήματος Τεχνητής Νοημοσύνης (TN). Προσέξτε ότι, πέρα από το ότι τα συστήματα TN παραισθησιάζονται, μπορεί να δώσουν παρόμοιες λύσεις σε διαφορετικούς φοιτητές, οπότε τα προγράμματά τους θα παρουσιάσουν ομοιότητες και θα μηδενιστούν.
 - Η έξοδος του προγράμματος θα πρέπει να περιλαμβάνει μόνο ό,τι φαίνεται στα Παραδείγματα Κλήσεων που παρατίθενται.

Η φλυαρία δεν επιβραβεύεται.

Χρήση του GitHub

Όπως αναφέρθηκε, το πρόγραμμά σας για να αξιολογηθεί θα πρέπει να αποθηκευθεί στο GitHub. Επιπλέον, θα πρέπει το GitHub να χρησιμοποιηθεί καθ' όλη τη διάρκεια της ανάπτυξής του. Θα εργαστείτε στο αποθετήριο που σας έχει αποδοθεί και:

- δεν θα κάνετε `fork` κάποιο άλλο αποθετήριο (όπως της εκφώνησης)

- θα εργαστείτε στο main branch
- δεν θα κάνετε pull και merge requests

Προσέξτε ότι *σίγουρα δεν θα ανεβάσετε στο GitHub απλώς την τελική λύση της εργασίας*. Στο GitHub θα πρέπει να φαίνεται το ιστορικό της συγγραφής του προγράμματος. Αυτό συνάδει και με τη φιλοσοφία του εργαλείου: λέμε “commit early, commit often”. Εργαζόμαστε σε ένα πρόγραμμα και κάθε μέρα, ή όποια στιγμή έχουμε κάνει κάποιο σημαντικό βήμα, αποθηκεύουμε την αλλαγή στο GitHub. Αυτό έχει σειρά ευεργετικών αποτελεσμάτων:

- Έχουμε πάντα ένα αξιόπιστο εφεδρικό μέσο στο οποίο μπορούμε να ανατρέξουμε αν κάτι πάει στραβά στον υπολογιστή μας (μας γλιτώνει από πανικούς του τύπου: ένα βράδυ πριν από την παράδοση ο υπολογιστής μας ή ο δίσκος του πνέει τα λοίσθια, και εμείς τι θα υποβάλουμε;).
- Καθώς έχουμε πλήρες ιστορικό των σημαντικών αλλαγών και εκδόσεων του προγράμματός μας, μπορούμε να επιστρέψουμε σε μία προηγούμενη αν συνειδητοποιήσουμε κάποια στιγμή ότι πήραμε λάθος δρόμο (μας γλιτώνει από πανικούς του τύπου: μα το πρωί δούλεψε σωστά, τι στο καλό έκανα και τώρα δεν παίζει τίποτε;).
- Καθώς φαίνεται η πρόοδός μας στο GitHub, επιβεβαιώνουμε στον διδάσκοντα ότι πραγματικά έχουμε εργαστεί μεθοδικά και δεν μας δόθηκε η λύση στο πιάτο (μας γλιτώνει από πανικούς του τύπου: καλά, πώς το έλυσες το πρόβλημα αυτό με τη μία, μόνος σου;).
- Αν δουλεύετε σε μία ομάδα που βεβαίως δεν είναι καθόλου η περίπτωση μας εδώ, μπορούν όλοι να εργάζονται στα ίδια αρχεία στο GitHub εξασφαλίζοντας ότι ο ένας δεν γράφει πάνω στις αλλαγές του άλλου. Παρά το ότι η εργασία αυτή είναι ατομική, καλό είναι να αποκτάτε τριβή με το εργαλείο git και την υπηρεσία GitHub, μιας και χρησιμοποιούνται ευρέως και όχι μόνο στη συγγραφή κώδικα.

Άρα επενδύστε λίγο χρόνο στην εκμάθηση των git / GitHub, των οποίων ο σωστός τρόπος χρήσης είναι μέσω γραμμής εντολών (command line), ή ειδικών προγραμμάτων (clients), ή μέσω ενοποίησης στο περιβάλλον ανάπτυξης (editor, IDE).

Όσον αφορά την αξιολόγηση της εργασίας, θα ληφθεί υπόψη ακριβώς η ιστορικότητα που θα έχει καταχωρηθεί στο GitHub. Δεν θα αξιολογηθούν εργασίες οι οποίες στο GitHub εμφανίζουν μόνο ένα commit, ή ελάχιστα commit. Η ιστορικότητα της εργασίας, όπως αποτυπώνεται στο GitHub, θα πρέπει να αντανακλά την πρόοδο στην εκπόνησή της από την αρχή της υλοποίησης μέχρι το τέλος της προθεσμίας.

Εργασίες που υποβάλλονται ως επιφοίτηση δεν θα βαθμολογούνται.

Παραδείγματα Κλήσεων

Κάθε παράδειγμα πιο κάτω μπορεί να τρέξει ακριβώς όπως είναι γραμμένο, πάνω στο σώμα κειμένων που σας δόθηκε. Η έξοδος που πρέπει να παραγάγει το καθένα βρίσκεται στον κατάλογο examples/ που το συνοδεύει, οπότε μπορείτε να συγκρίνετε τη δική σας απευθείας με αυτήν:

```
python synthid.py --generate --key rwa-synthid-2026 --seed 42 --length 12 > mine.json
diff mine.json examples/example_generate.json
```

Παράδειγμα 1: Παραγωγή Υδατογραφημένου Κειμένου

Αν ο χρήστης δώσει:

```
python synthid.py --generate --key rwa-synthid-2026 --seed 42 --length 12
```

το πρόγραμμά σας θα παραγάγει την ακόλουθη έξοδο:

```
{
  "mode": "generate",
  "seed": 42,
  "watermarked": true,
  "entropy": "high",
  "layers": 30,
  "window": 4,
  "sampler": "layered",
  "tokens": [
    "w063",
    "w248",
    "w784",
    "w964",
    "w488",
    "w927",
    "w299",
    "w743",
    "w095",
    "w472",
    "w167",
    "w071"
  ]
}
```

Τα πρώτα τέσσερα τεμάχια είναι η προτροπή, κληρωμένα απευθείας από το λεξιλόγιο· τα υπόλοιπα οκτώ είναι αυτά που έδωσε το τουρνουά.

Παράδειγμα 2: Το Ίδιο Κείμενο, Χωρίς Υδατογράφημα

Αν ο χρήστης δώσει:

```
python synthid.py --generate --key rwa-synthid-2026 --seed 42 --length 12 \
  --no-watermark
```

η έξοδος έχει την ίδια μορφή, με **"watermarked": false**, και αυτά τα τεμάχια:

```
"tokens": [ "w063", "w248", "w784", "w964", "w661", "w546",
  "w284", "w632", "w887", "w441", "w937", "w700" ]
```

Ολόκληρη η έξοδος βρίσκεται στο `examples/example_generate_plain.json`.

Χρησιμοποιούμε τον ίδιο σπόρο, οπότε παίρνουμε την ίδια προτροπή: τα πρώτα τέσσερα τεμάχια είναι ταυτόσημα με του Παραδείγματος 1. Από το πέμπτο και μετά διαφέρουν, επειδή αυτή η εκτέλεση κληρώνει κατευθείαν από το μοντέλο αντί από το τουρνουά.

Παράδειγμα 3: Ο Δειγματολήπτης Νοκ-άουτ

Αν ο χρήστης δώσει:

```
python synthid.py --generate --key rwa-synthid-2026 --seed 7 --length 8 \
--sampler knockout --layers 4
```

η έξοδος καταγράφει τον δειγματολήπτη και τον αριθμό επιπέδων με τα οποία έτρεξε:

```
"layers": 4,
"sampler": "knockout",
"tokens": ["w566", "w899", "w927", "w742", "w904", "w705", "w077", "w247"]
```

Ολόκληρη η έξοδος βρίσκεται στο `examples/example_generate_knockout.json`.

Έχουμε τέσσερα επίπεδα, οπότε κάθε ένα από τα τέσσερα παραγόμενα τεμάχια κόστισε $2^4 = 16$ κληρώσεις υποψηφίων. Με τα προεπιλεγμένα 30 επίπεδα η ίδια εντολή θα χρειαζόταν πάνω από ένα δισεκατομμύριο κληρώσεις για κάθε τεμάχιο.

Παράδειγμα 4: Ανίχνευση Υδατογραφήματος

Αν ο χρήστης δώσει:

```
python synthid.py --detect --key rwa-synthid-2026 --alpha 0.01 \
--calibration corpus/calibration_T200.txt \
--documents corpus/watermarked_T200.txt
```

το πρόγραμμά σας θα παραγάγει έξοδο με μία εγγραφή για κάθε ένα από τα 200 κείμενα. Ολόκληρη η έξοδος βρίσκεται στο `examples/example_detect_watermarked.json`. οι πρώτες και οι τελευταίες γραμμές του είναι:

```
{
  "mode": "detect",
  "alpha": 0.01,
  "layers": 30,
  "window": 4,
  "calibration": {
    "documents": 1000,
    "threshold": 3031
  },
  "documents": [
    {
      "index": 0,
      "scorable": 196,
      "numerator": 3192,
      "score": 0.5428571428571428,
      "flagged": 1
    },
    ... 199 more ...
  ],
  "flagged": 200
}
```

Το κατώφλι, 3031, προήλθε από τα χίλια μη υδατογραφημένα κείμενα βαθμονόμησης: είναι ο μικρότερος αριθμητής του οποίου η ουρά βρίσκεται μέσα στο $\alpha = 0,01$ αυτών. Κάθε κείμενο ανιχνεύεται όταν ο αριθμητής του φτάνει το κατώφλι. Και τα 200 το φτάνουν, οπότε το ποσοστό αληθώς θετικών σε αυτό το μήκος είναι 200/200. Στα 200 τεμάχια το υδατογράφημα είναι αναμφισβήτητο.

Παράδειγμα 5: Ο Ίδιος Ανιχνευτής σε Μη Υδατογραφημένο Κείμενο

Αν ο χρήστης δώσει:

```
python synthid.py --detect --key rwa-synthid-2026 --alpha 0.01 \
--calibration corpus/calibration_T200.txt \
--documents corpus/unwatermarked_T200.txt
```

το κατώφλι είναι το ίδιο 3031, επειδή εξαρτάται μόνο από τα κείμενα βαθμονόμησης, και το flagged βγαίνει 3. Η πλήρης έξοδος βρίσκεται στο `examples/example_detect_unwatermarked.json`.

Τρία από 200 είναι το ποσοστό ψευδώς θετικών 1,5% έναντι α 1%. Δεν μπορούμε να πετύχουμε το α ακριβώς, όπως εξηγήσαμε όταν συζητήσαμε τον τύπο του κατωφλίου. Στο ίδιο το σύνολο κειμένων βαθμονόμησης το $k = 3031$ ανιχνεύει 9 κείμενα από 1000, δηλαδή 0,9%. ο επόμενος αριθμητής προς τα κάτω ανιχνεύει 11, πάνω από τον όριο των 10, οπότε κανένα κατώφλι δεν δίνει ακριβώς 1%. Και αυτά τα 200 κείμενα είναι διαφορετική παρτίδα από τα 1000 στα οποία ορίστηκε το κατώφλι, οπότε το ποσοστό τους σκορπίζεται γύρω από εκείνη την τιμή αντί να την αναπαράγει ακριβώς. Αυτή είναι αναμενόμενη συμπεριφορά.

Παράδειγμα 6: Ανίχνευση με Φίλτρο Bloom

Αν ο χρήστης δώσει:

```
python synthid.py --detect --key rwa-synthid-2026 --alpha 0.01 \
--calibration corpus/calibration_T200.txt \
--documents corpus/watermarked_T200.txt --bloom-bits 512
```

παίρνουμε τώρα ένα πλήθος `false_skips` που αναφέρει τα ψευδώς θετικά του φίλτρου Bloom. Οι πρώτες και οι τελευταίες γραμμές της εξόδου είναι τώρα:

```
{
  "mode": "detect",
  "alpha": 0.01,
  "layers": 30,
  "window": 4,
  "calibration": {
    "documents": 1000,
    "threshold": 2765
  },
  "documents": [
    {
      "index": 0,
      "scorable": 175,
      "numerator": 2851,
      "score": 0.543047619047619,
      "flagged": 1,
      "false_skips": 21
    }
  ]
}
```

```

    },
    ... 199 more ...
  ],
  "flagged": 145,
  "bloom": {
    "bits": 512,
    "hashes": 3
  }
}

```

Ολόκληρη η έξοδος βρίσκεται στο `examples/example_detect_bloom.json`.

Το ίδιο κείμενο βαθμολογήθηκε σε 196 θέσεις στο Παράδειγμα 4 και βαθμολογείται σε 175 εδώ: τα 512 bits είναι πολύ λίγα, οπότε το φίλτρο αναφέρει συνεχώς ως ήδη ειδικμένες θέσεις που ποτέ δεν ήταν. Σε όλα τα 200 κείμενα αυτό πετάει 4292 θέσεις από τις 39200, περίπου το ένα ένατο, και το ποσοστό αληθώς θετικών πέφτει από 200/200 σε 145/200.

Βλέπουμε ότι μετακινήθηκε και το κατώφλι, από 3031 σε 2765: τα κείμενα βαθμονόμησης βαθμολογούνται μέσα από το ίδιο φίλτρο, οπότε οι αριθμητές τους μικραίνουν όπως ακριβώς και των κρινόμενων κειμένων. Παρόλο που βαθμονομούμε τον ανιχνευτή, όπως οφείλουμε, αυτό δεν μπορεί να σώσει τις τελικές ετυμηγορίες ανά κείμενο. Έχει ενδιαφέρον να δούμε το γιατί.

Χωρίς φίλτρο, κάθε κείμενο εδώ έχει τις ίδιες 196 βαθμολογήσιμες θέσεις, οπότε η κατάταξη των κειμένων κατά αριθμητή είναι η ίδια με την κατάταξη σύμφωνα με τη βαθμολογία κάθε κειμένου. Με φίλτρο, κάθε κείμενο κρατά διαφορετικό αριθμό θέσεων (από 163 έως 186 σε αυτά τα 200) και ο αριθμητής του μειώνεται αντίστοιχα. Το κατώφλι είναι ένας αριθμός μόνο, που δεν γνωρίζει τίποτε για το πόσες θέσεις κράτησε κάθε συγκεκριμένο κείμενο, οπότε αρχίζει να ταξινομεί τα κείμενα κατά το πόσες θέσεις διατηρήθηκαν και όχι κατά το πόσο ισχυρό είναι το υδατογράφημά τους. Σε αυτήν την εκτέλεση το κείμενο 163 βαθμολογείται 0,5503 και διαφεύγει, ενώ το κείμενο 160 βαθμολογείται 0,5241 και εντοπίζεται, επειδή κράτησε εννέα θέσεις παραπάνω. Τα κείμενα που πέρασαν έχουν κατά μέσο όρο 176 επιζώσες θέσεις: αυτά που δεν πέρασαν 170, με πρακτικά την ίδια βαθμολογία.

Όπως είπαμε, ένας αριθμητής είναι συγκρίσιμος μόνο ανάμεσα σε κείμενα που βαθμολογήθηκαν πάνω στον ίδιο αριθμό θέσεων. Εδώ βλέπουμε τι συμβαίνει όταν η συνθήκη αυτή δεν ισχύει.

Παράδειγμα 7: Ένα Πιο Ευρύχωρο Φίλτρο Bloom

Αν ο χρήστης δώσει την ίδια εντολή με φίλτρο δεκαέξι φορές μεγαλύτερο:

```

python synthid.py --detect --key rwa-synthid-2026 --alpha 0.01 \
    --calibration corpus/calibration_T200.txt \
    --documents corpus/watermarked_T200.txt --bloom-bits 8192

```

οι πρώτες και οι τελευταίες γραμμές της εξόδου γίνονται:

```

{
  "mode": "detect",
  "alpha": 0.01,
  "layers": 30,
  "window": 4,
  "calibration": {
    "documents": 1000,

```

```

    "threshold": 3031
  },
  "documents": [
    {
      "index": 0,
      "scorable": 196,
      "numerator": 3192,
      "score": 0.5428571428571428,
      "flagged": 1,
      "false_skips": 0
    },
    ... 199 more ...
  ],
  "flagged": 200,
  "bloom": {
    "bits": 8192,
    "hashes": 3
  }
}

```

Ολόκληρη η έξοδος βρίσκεται στο `examples/example_detect_bloom_roomy.json`.

Αυτές είναι οι ίδιες 196 θέσεις, ο ίδιος αριθμητής και η ίδια ετυμηγορία με το Παράδειγμα 4, που δεν χρησιμοποίησε καθόλου φίλτρο—και επιτυγχάνονται με 8192 bits αντί για ένα σύνολο που μεγαλώνει μαζί με το κείμενο. Σε όλα τα 200 κείμενα μόνο 5 θέσεις παρακάμπτονται εσφαλμένα, 195 από τα κείμενα δεν χάνουν απολύτως τίποτε, και κανένα κείμενο δεν χάνει πάνω από μία. Το κατώφλι επιστρέφει στο 3031, και ανιχνεύονται πάλι και τα 200. Μάλιστα, μπορείτε να ελέγξετε ότι με φίλτρο Bloom 2048 bits ανιχνεύονται και τα 200, με κλάσμα της μνήμης που θα χρειαζόμασταν με ένα σύνολο.

Αναφορές

- [1] Sumanth Dathathri et al. “Scalable watermarking for identifying large language model outputs”. In: *Nature* 634.8035 (2024), pp. 818–823.

Καλή επιτυχία!